

Machine Learning

Week 6 Lecture

Dr. Muhammad Ahmed Raza

Contents

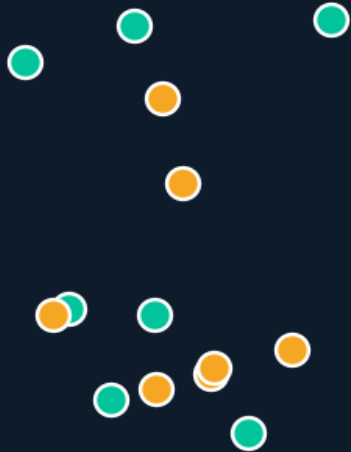
- Intro to Unsupervised Learning
- Why clustering?
- K-means
 - X-means
 - K-means++

What is Unsupervised Learning?

Finding structure in data — without being told the answers

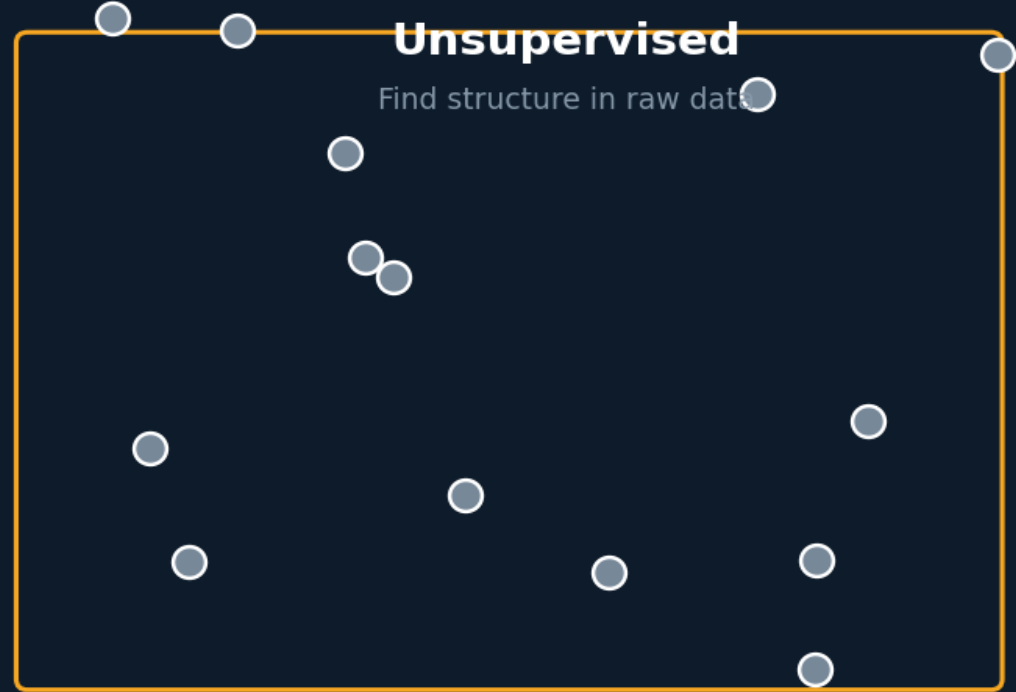
Supervised

Labels tell us the answer



Unsupervised

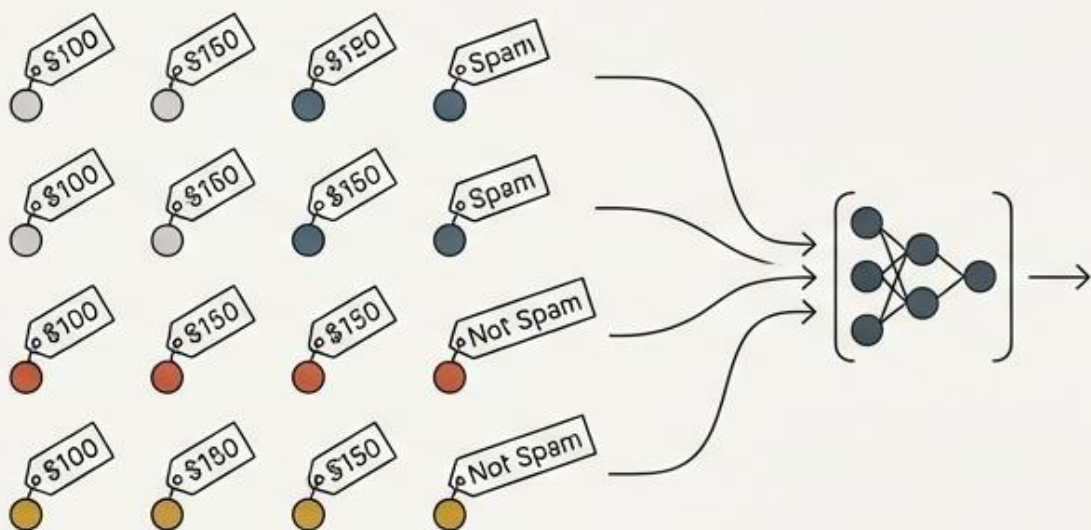
Find structure in raw data



💡 In supervised learning, labels guide the model. In unsupervised learning, the model must discover the structure on its own.

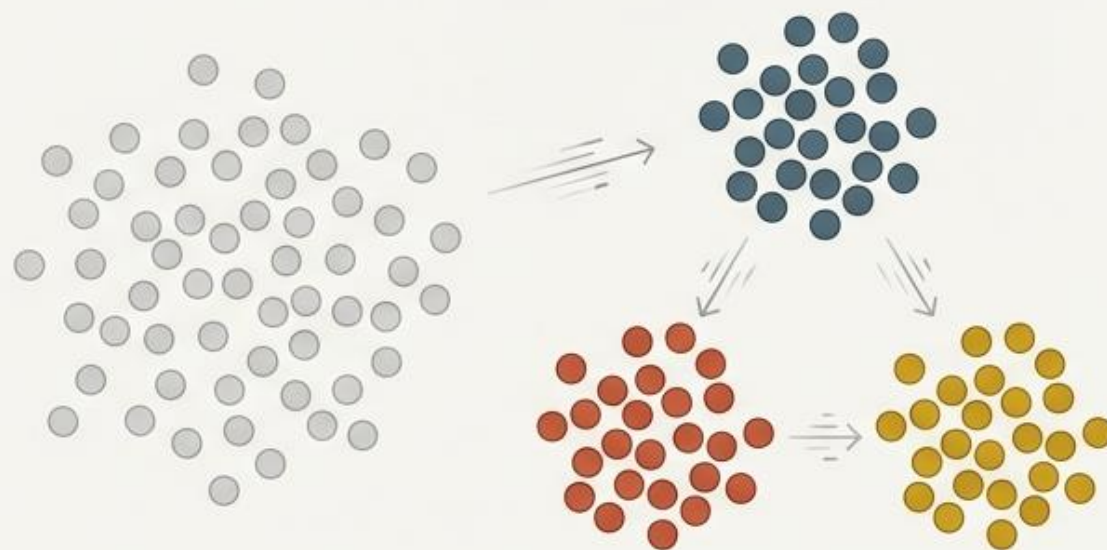
Unsupervised learning discovers structure without labels

Supervised



Supervised Learning: Maps features to known explicit labels (e.g., price prediction, spam detection).

Unsupervised



Unsupervised Learning: Operates on features alone to automatically group data. Points within a cluster are similar; points in different clusters are distinct.

Applications: Market segmentation | Anomaly detection | Document grouping | Image compression

Why Clustering?

Grouping similar things together — without labels



Customer Segmentation

Group buyers by purchase behaviour — without knowing in advance how many types exist.



Genomics

Find genes with similar expression patterns across thousands of samples.



Document Grouping

Organize news articles into topics with no pre-defined categories.



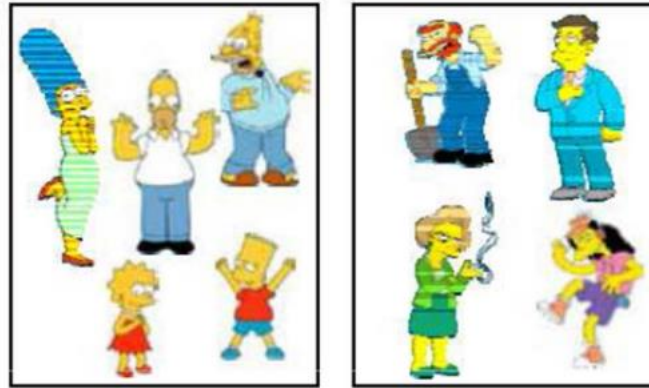
Anomaly Detection Prep

Cluster normal behaviour first — outliers fall outside every cluster.

Clustering algorithms

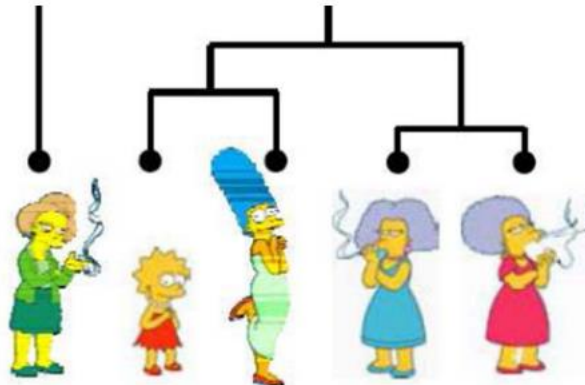
- Partition algorithms (Flat)

- K-means
- Mixture of Gaussian
- Spectral Clustering



- Hierarchical algorithms

- Bottom up – agglomerative
- Top down – divisive



The K-Means Algorithm

An iterative, assignment-update loop



1 Choose K random centroids

2 Assign each point to nearest centroid

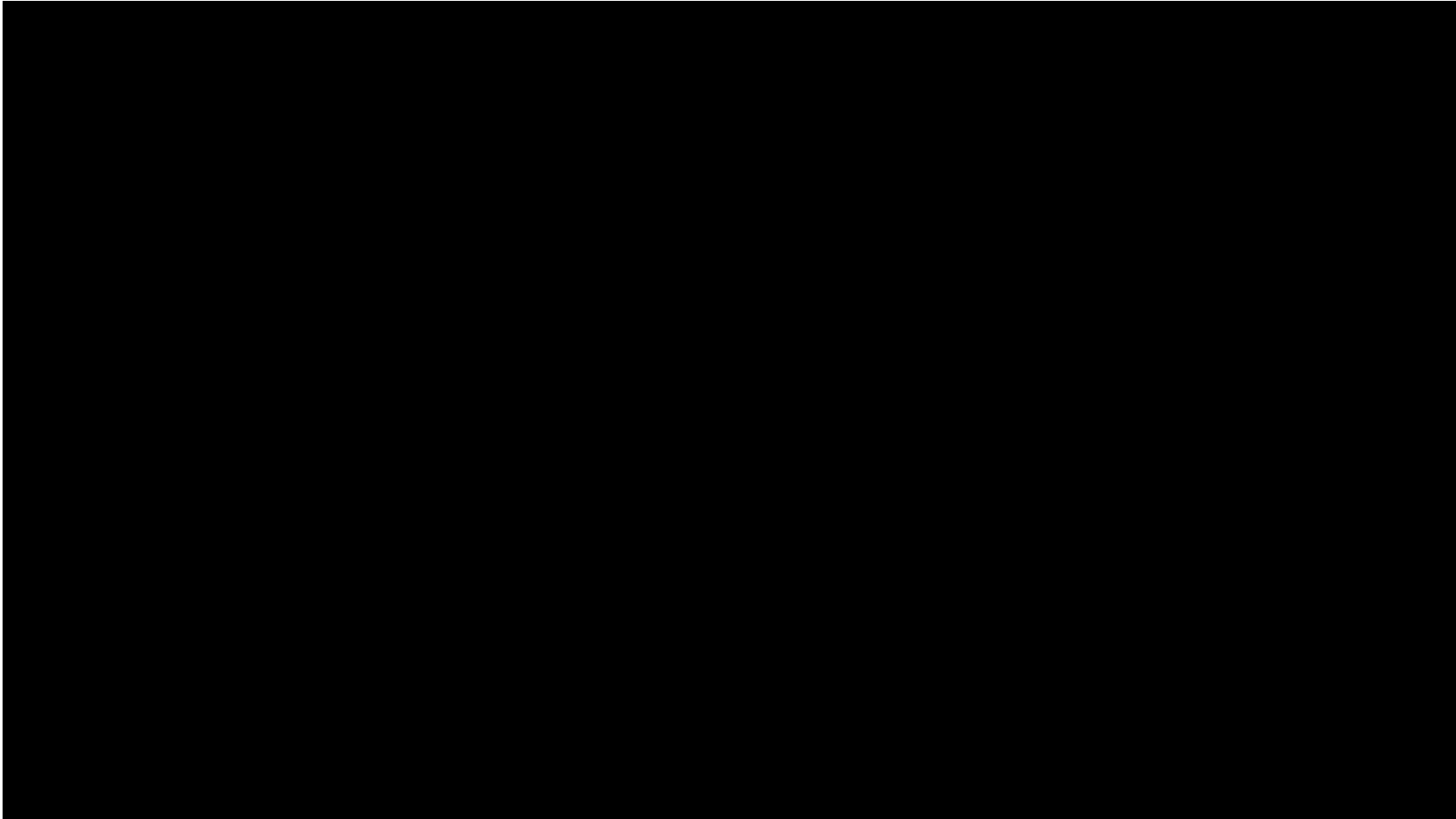
3 Recompute centroids as cluster means

4 Repeat until centroids stop moving



Convergence guaranteed — but may reach a local optimum. Run multiple times with different initializations.

K-means demo



K-means Online demo

- <https://www.naftaliharris.com/blog/visualizing-k-means-clustering/>

The algorithm minimizes the Within-Cluster Sum of Squares

$$J = \sum || \mathbf{x}_i - \mu_k ||^2$$

↑
The Objective
Function
(Inertia or WCSS)

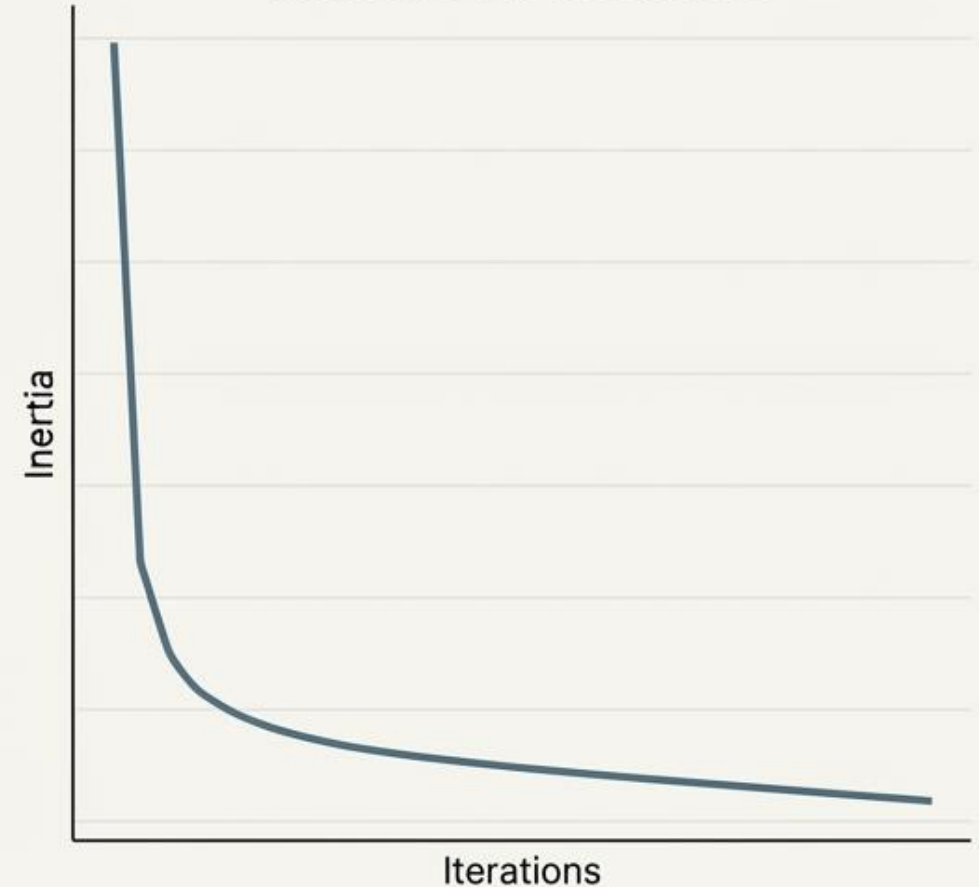
↑
The data
point

↑
The assigned
centroid for
that point

↑
Squared
Euclidean
distance

K-Means is an optimization problem. The algorithm constantly attempts to minimize the squared distances between points and their respective cluster centers.

Inertia over Iterations



The Math Behind K-Means

Objective

$$\min_{\mu} \min_C \sum_{i=1}^k \sum_{x \in C_i} |x - \mu_i|^2$$

1. Fix μ , optimize C :

$$\min_C \sum_{i=1}^k \sum_{x \in C_i} |x - \mu_i|^2 = \min_c \sum_i^n |x_i - \mu_{x_i}|^2$$

Step 1 of kmeans

2. Fix C , optimize μ :

$$\min_{\mu} \sum_{i=1}^k \sum_{x \in C_i} |x - \mu_i|^2$$

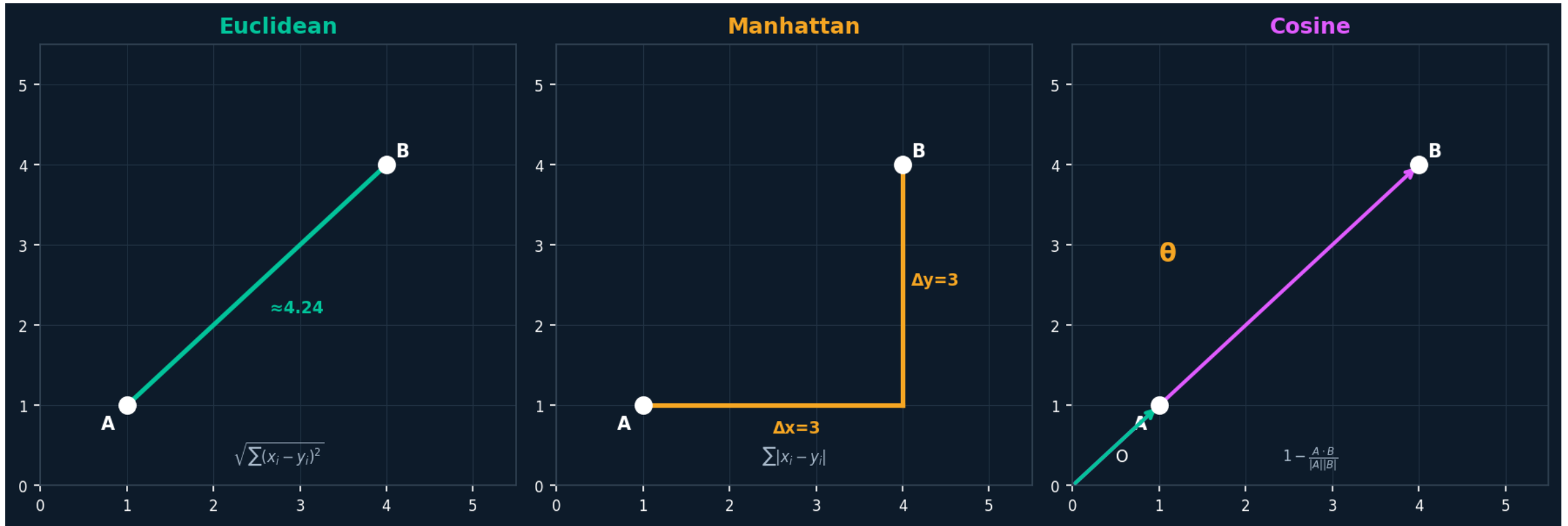
- Take partial derivative of μ_i and set to zero, we have

$$\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

Step 2 of kmeans

Distance Metrics

How we measure 'closeness' determines what clusters form



Euclidean

Straight-line distance. Sensitive to scale.
Default choice.

Manhattan

Sum of absolute differences. More robust to outliers.
Grid-like paths.

Cosine

Measures angle, not magnitude. Used in text/NLP where direction matters.

Metric Choice Changes Everything

Same dataset, same $K=3$, different distance $\rightarrow$ different clusters

Same Data, Different Distance Metrics $\rightarrow$ Different Clusters

Euclidean Distance

Manhattan Distance

Cosine Similarity



Key insight: The choice of distance metric encodes your assumption about what 'similar' means. Always ask: does straight-line distance make sense for my features? Are my features on the same scale? Is direction more important than magnitude?

Inertia — Measuring Cluster Quality

The objective function K-Means minimizes

Inertia = Sum of squared distances to centroid

Lower inertia → tighter, more compact clusters



The Formula

$$SSE = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

*Sum of squared distances
from each point to its centroid*

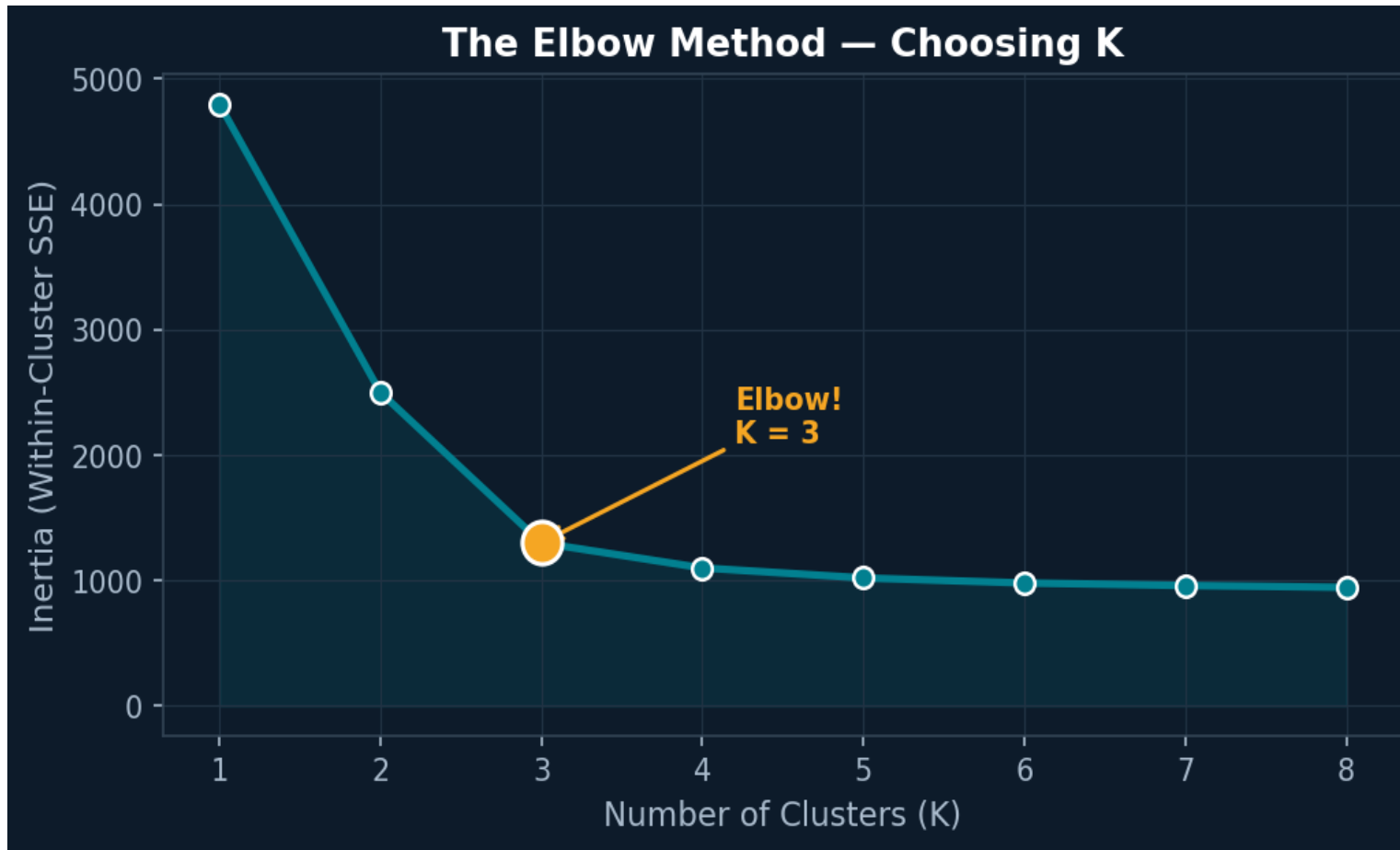
↓ **Lower inertia** = tighter, more compact clusters

↑ **More clusters K** = always lower inertia (trivial limit)

⚠ **Inertia alone** cannot tell you the right K

Choosing K — The Elbow Method

Plot inertia vs K; pick the bend in the curve



① Start with $K=1$ and increase up to ~ 10

② Record the inertia for each K after fitting

③ Plot K on x-axis, inertia on y-axis

④ Look for the 'elbow' — where marginal gain flattens

⑤ That K is your best trade-off between fit and complexity

⚠ The elbow is a heuristic, not a law. Use domain knowledge + the elbow together. Sometimes there is no clear elbow.

K-Means Limitations

Know when it fails before you deploy it



You must choose K

K-Means requires you to specify the number of clusters in advance. The elbow method helps — but it's a heuristic.



Assumes spherical clusters

The algorithm minimizes Euclidean distance, so it expects round, equally-sized blobs. Elongated or irregular shapes fail.



Sensitive to initialization

Different random starting centroids can yield different final clusters. Use K-Means++ or multiple restarts.



Sensitive to scale

Features with larger ranges dominate the distance. Always standardize (zero mean, unit variance) before clustering.



Local optima

K-Means converges to a local minimum, not necessarily global. The objective J is non-convex.

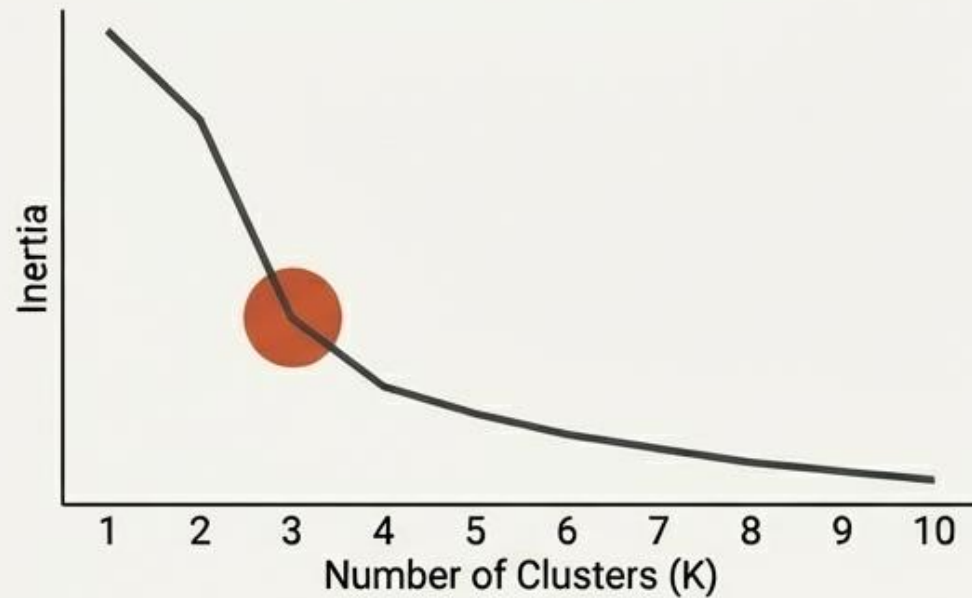


High dimensions degrade

In high-dimensional spaces, distances become meaningless (curse of dimensionality). Reduce dimensions first (PCA).

Challenge 1: The algorithm cannot determine the optimal number of clusters

The Elbow Method



The Elbow Method: Plot the percentage of explained variance (Inertia) against K. The optimal K is at the "elbow" where the marginal gain drops off.

Limitation: Highly subjective; the angle is often not well-defined.

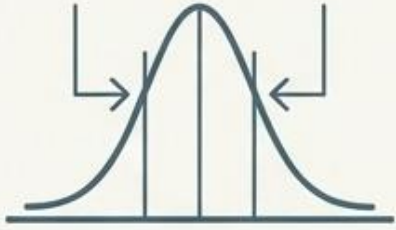
X-means Clustering

- Start with a small number of clusters (often $K = 2$).
- Run K-means to form clusters.
- Try splitting each cluster into two sub-clusters.
- Evaluate whether the split improves the model using a statistical criterion like Bayesian Information Criterion (BIC).

$$BIC = -2 \ln(L) + k \ln(n)$$

- Accept the split if BIC improves.
- Repeat until no further beneficial splits are found

Advanced algorithmic heuristics for dynamically determining K



X-means Clustering

Repeatedly attempts subdivision and keeps the best splits until an Akaike (AIC) or Bayesian Information Criterion (BIC) is reached.



Information-Theoretic "Jump" Method

Transforms the distortion curve by a negative power chosen based on dimensionality. The largest sudden "jump" represents the most efficient and robust K.



Gap Statistics

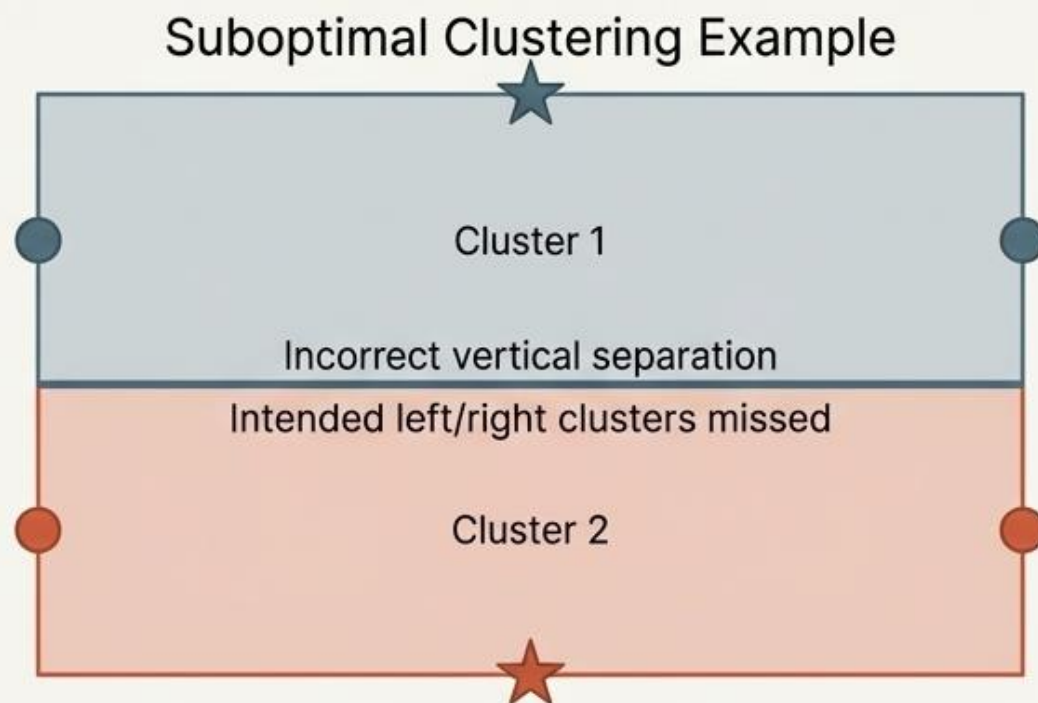
Measures how far the pooled within-cluster sum of squares falls below the expected sum of squares under a null reference distribution (random data lacking any clusters).

Challenge 2: Random initialization leads to poor local minima

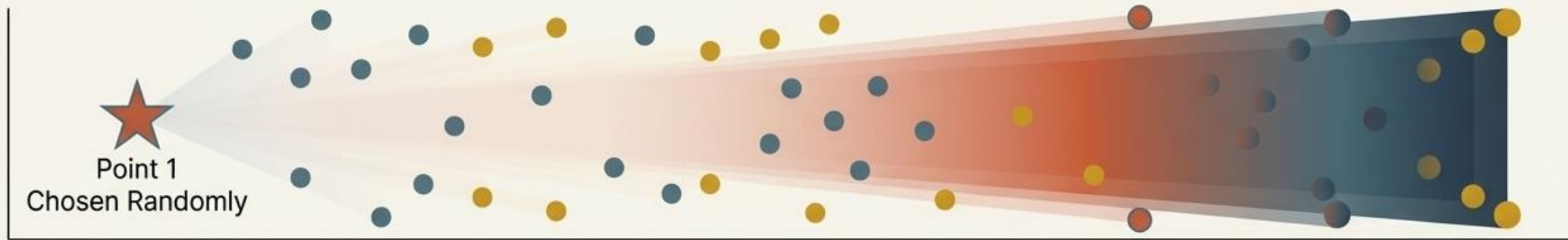
- Standard K-Means is highly sensitive to initial centroid placement.
- If initializing k centroids randomly across k true clusters, the probability of placing exactly one centroid into each unique cluster is approximately:

$$\frac{k!}{k^k}$$

- This probability drops to zero incredibly fast as dimensions increase.
- Result: It is virtually guaranteed that random initialization will result in some true clusters getting multiple centroids while others are ignored entirely.



The Solution: K-Means++ guarantees an $O(\log k)$ competitive approximation



Instead of pure randomness, K-Means++ probabilistically forces initial centroids to spread out.

Step 1: Choose the first centroid uniformly at random from the data points.

Step 2: Compute $D(x)$, the distance between every data point and its nearest existing centroid.

Step 3: Choose the next centroid from the remaining data points with a probability proportional to its squared distance: $D(x)^2$.

Result: Dramatically lowers computation time to converge and significantly reduces final error by ensuring optimal seed distribution.

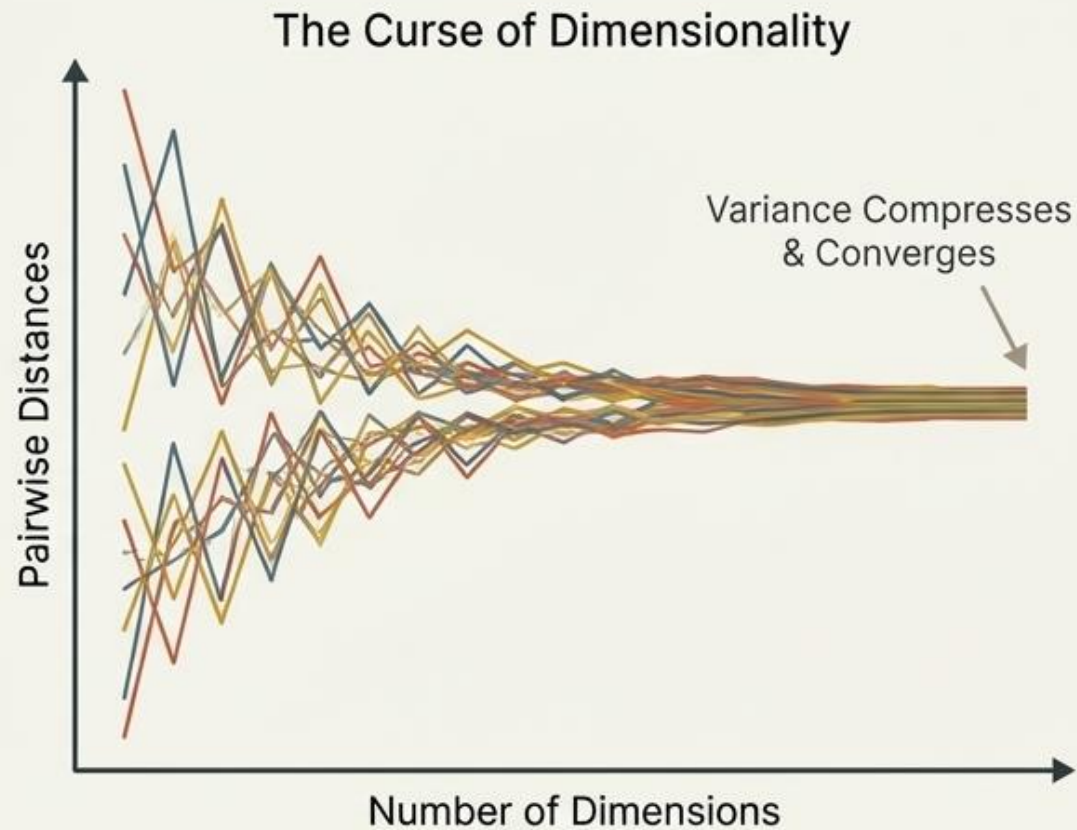
K-means++

- Choose the first centroid randomly from the dataset.
- For every remaining data point, compute its distance from the nearest chosen centroid.
- Select the next centroid with probability proportional to the squared distance D^2 .

$$P(x_i) = \frac{D_i^2}{\sum_{j=1}^n D_j^2}$$

- Points far from existing centroids are more likely to be chosen.
- Repeat until K centroids are selected.
- Run the standard K-means algorithm using these centroids.

Challenge 3: Distance becomes meaningless in high dimensions

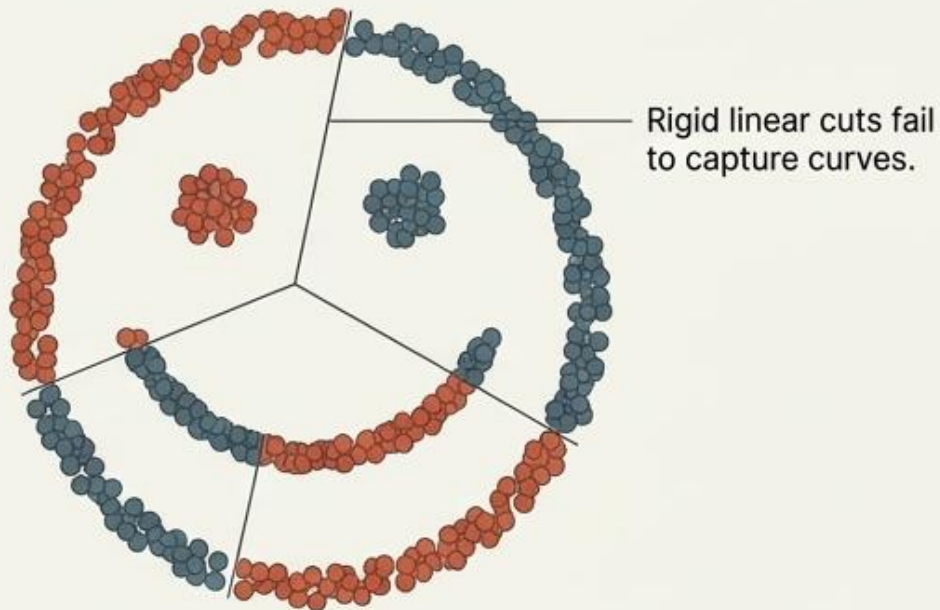


The Curse of Dimensionality

- As dimensions increase, the standard deviation in distance between examples shrinks relative to the mean distance.
- A distance-based similarity measure ultimately converges to a constant value between ANY given examples.
- The result: K-Means becomes 'blind' and is completely unable to distinguish between points.

Overcoming shape and dimension limits with Spectral Clustering

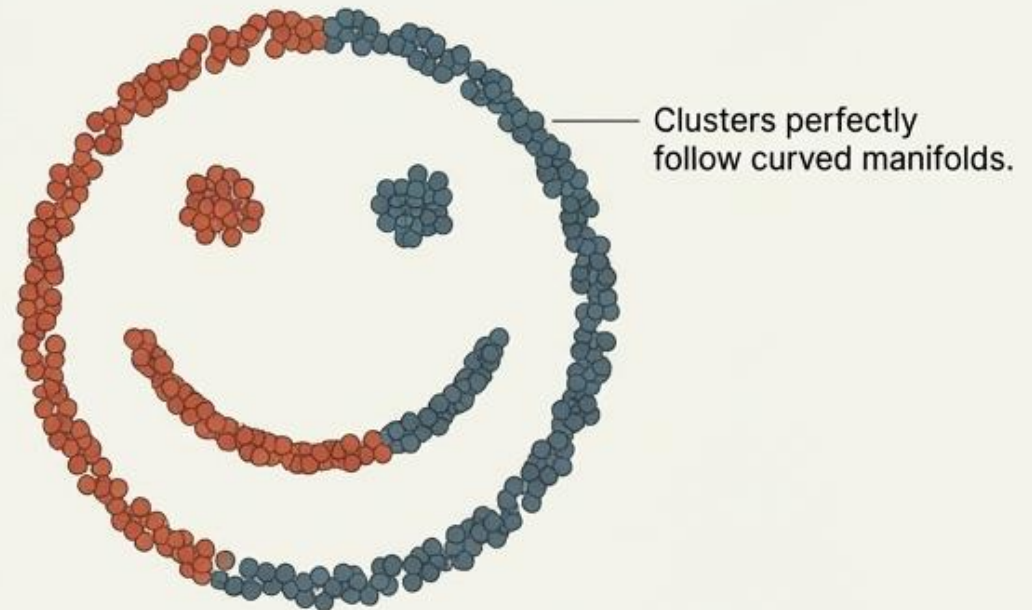
Standard K-Means Failure



The Shape Problem: K-Means assumes hyper-spherical clusters and struggles heavily with varying densities, curved shapes, or intersecting geometries. Centroids can also be drastically dragged by severe outliers.

The Fix - Outlier Clipping: Always pre-process data to remove or clip extreme outliers before clustering to protect centroid gravity.

Spectral Clustering Success



The Fix - Spectral Clustering: Adds mathematical pre-clustering steps to bypass standard limitations.

1. Uses PCA to project all data points into a lower-dimensional subspace, completely avoiding the curse of dimensionality.
2. Clusters the data in that continuous subspace to capture curved manifolds.

Executive Summary: Deploying K-Means in Production

Pros

- Simple to implement and interpret
- Scales gracefully to massive datasets
- Mathematically guarantees convergence
- Smoothly adapts to new examples in live environments

Cons

- Requires manual tuning of K
- Highly sensitive to structural outliers
- Biased towards equally sized hyper-spheres
- Succumbs to the curse of dimensionality in high-feature spaces

The Golden Rules

1. Always use K-Means++ seeding to ensure stability.
2. Clip or remove outliers before fitting to protect centroid gravity.
3. Apply dimensionality reduction (PCA or Spectral) if working with high-dimensional feature spaces.